SECURITY LOG INVESTIGATION

Python mini-project • Defensive log analysis • Beginner friendly

Project objective

Analyze a local web-server log, identify HTTP 401/403 failures and simple suspicious-path indicators, count repeated failures by IP address, and export flagged records to a CSV report.

Tools

Python 3 standard library only: argparse, csv, re, collections, datetime, pathlib. No paid tools or external packages are required.

Important

Use only logs you own or are authorized to examine. The indicators in this demo are simple review hints, not proof of an attack. The example IP addresses are documentation-only sample addresses.

1. How the project works

1) Reads each line from a web-server log file. 2) Extracts timestamp, IP, request path, HTTP status, and response size. 3) Flags 401/403 responses and selected suspicious path strings. 4) Marks repeated failures when an IP reaches the configured threshold. 5) Writes flagged records to a CSV file for review.

2. Source code (security_log_investigation.py)

```
#!/usr/bin/env python3
"""
Security Log Investigation (defensive educational project)
Reads a local web-server style log file and reports:
- failed authentication attempts (401/403)
- repeated failures from the same IP
- suspicious request paths (basic keyword indicators)
- a CSV report for review

Input format example:
203.0.113.10 - - [15/Oct/2026:09:10:00 +0000] "GET /login HTTP/1.1" 401 234
"""

import argparse
import csv
import re
from collections import Counter, defaultdict
from datetime import datetime
from pathlib import Path

LOG_PATTERN = re.compile(
    r'(?P<ip>\S+)\s+\S+\s+\S+\s+\[(?P<time>[^\]]+)\]\s+'
    r'"(?P<request>[^"]*)"\s+(?P<status>\d{3})\s+(?P<size>\S+)'
)

SUSPICIOUS_PATH_TERMS = (
    "../", "/.env", "/wp-admin", "/phpmyadmin", "passwd", "config"
)

def parse_line(line):
    """Parse one common/combined web-server log line; return None if unmatched."""
    match = LOG_PATTERN.search(line)
    if not match:
        return None

    request_parts = match.group("request").split()
    method = request_parts[0] if request_parts else "-"
    path = request_parts[1] if len(request_parts) > 1 else "-"
    try:
        timestamp = datetime.strptime(
            match.group("time").split()[0], "%d/%b/%Y:%H:%M:%S"
        ).isoformat(sep=" ")
    except ValueError:
        timestamp = match.group("time")

    return {
        "ip": match.group("ip"),
        "timestamp": timestamp,
        "method": method,
        "path": path,
        "status": int(match.group("status")),
        "size": match.group("size"),
    }

def investigate(log_path, report_path, threshold=5):
    events = []
    parse_errors = 0

    with open(log_path, "r", encoding="utf-8", errors="replace") as log_file:
        for line_number, line in enumerate(log_file, start=1):
            record = parse_line(line)
            if record is None:
                parse_errors += 1
                continue

            record["line_number"] = line_number
            flags = []

            if record["status"] in (401, 403):
                flags.append("authentication_or_access_failure")
```

```
            if any(term.lower() in record["path"].lower()
                   for term in SUSPICIOUS_PATH_TERMS):
                flags.append("suspicious_path_indicator")

            record["flags"] = ";".join(flags) if flags else "none"
            events.append(record)

    failures_by_ip = Counter(
        event["ip"] for event in events if event["status"] in (401, 403)
    )

    for event in events:
        if failures_by_ip[event["ip"]] >= threshold and event["status"] in (401, 403):
            existing = event["flags"].split(";")
            if "repeated_failures_threshold_met" not in existing:
                existing.append("repeated_failures_threshold_met")
            event["flags"] = ";".join(existing)

    flagged = [event for event in events if event["flags"] != "none"]

    fields = [
        "line_number", "timestamp", "ip", "method",
        "path", "status", "size", "flags"
    ]
    with open(report_path, "w", newline="", encoding="utf-8") as csv_file:
        writer = csv.DictWriter(csv_file, fieldnames=fields)
        writer.writeheader()
        writer.writerows(flagged)

    print("=== Security Log Investigation ===")
    print(f"Parsed log entries: {len(events)}")
    print(f"Unparsed lines: {parse_errors}")
    print(f"Flagged entries: {len(flagged)}")
    print(f"CSV report saved to: {report_path}")
    print("\nFailed access attempts by IP:")
    if failures_by_ip:
        for ip, count in failures_by_ip.most_common():
            marker = "  <-- review" if count >= threshold else ""
            print(f"  {ip}: {count}{marker}")
    else:
        print("  No HTTP 401/403 responses found.")

def main():
    parser = argparse.ArgumentParser(
        description="Analyze a local web-server log for basic defensive indicators."
    )
    parser.add_argument("logfile", help="Path to a local text log file")
    parser.add_argument("--report", default="security_report.csv",
                        help="CSV output path (default: security_report.csv)")
    parser.add_argument("--threshold", type=int, default=5,
                        help="Flag IPs with at least this many 401/403 responses")
    args = parser.parse_args()

    if args.threshold < 1:
        parser.error("--threshold must be at least 1")
    if not Path(args.logfile).is_file():
        parser.error(f"log file not found: {args.logfile}")

    investigate(args.logfile, args.report, args.threshold)

if __name__ == "__main__":
    main()
```

3. Sample input (sample_access.log)

```
203.0.113.10 - - [15/Oct/2026:09:10:00 +0000] "GET /login HTTP/1.1" 401 234
203.0.113.10 - - [15/Oct/2026:09:10:04 +0000] "POST /login HTTP/1.1" 401 245
203.0.113.10 - - [15/Oct/2026:09:10:08 +0000] "POST /login HTTP/1.1" 403 245
203.0.113.10 - - [15/Oct/2026:09:10:12 +0000] "POST /login HTTP/1.1" 401 245
203.0.113.10 - - [15/Oct/2026:09:10:16 +0000] "POST /login HTTP/1.1" 401 245
198.51.100.24 - - [15/Oct/2026:09:12:00 +0000] "GET /index.html HTTP/1.1" 200 1200
192.0.2.40 - - [15/Oct/2026:09:13:00 +0000] "GET /.env HTTP/1.1" 404 120
```

4. Run the project

```
python security_log_investigation.py sample_access.log

# Choose a report filename and threshold:
python security_log_investigation.py sample_access.log --report report.csv --threshold 5
```

5. Example output

```
=== Security Log Investigation ===
Parsed log entries: 7
Unparsed lines: 0
Flagged entries: 6
CSV report saved to: security_report.csv

Failed access attempts by IP:
  203.0.113.10: 5  <-- review
  No HTTP 401/403 responses found.
```

Note: The exact flagged count depends on the sample and code version. Review the generated CSV for the authoritative list of flagged entries.

6. Suggested project report sections

Problem statement: Server logs can contain repeated failed requests and unusual paths that are difficult to review manually.

Methodology: Parse log records, apply transparent rules, group failures by IP, and export flagged rows.

Limitations: A shared IP can represent many users; legitimate requests may be flagged; the script does not establish intent, correlate multiple log sources, or replace a SIEM or human investigation.

Future enhancements: Add date-range filtering, configurable rules, summary charts, JSON support, and unit tests using synthetic logs.